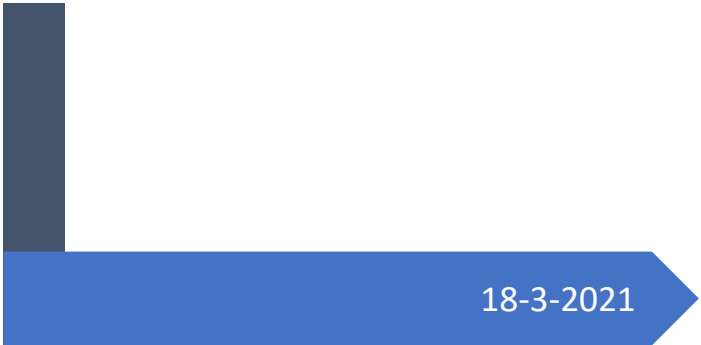


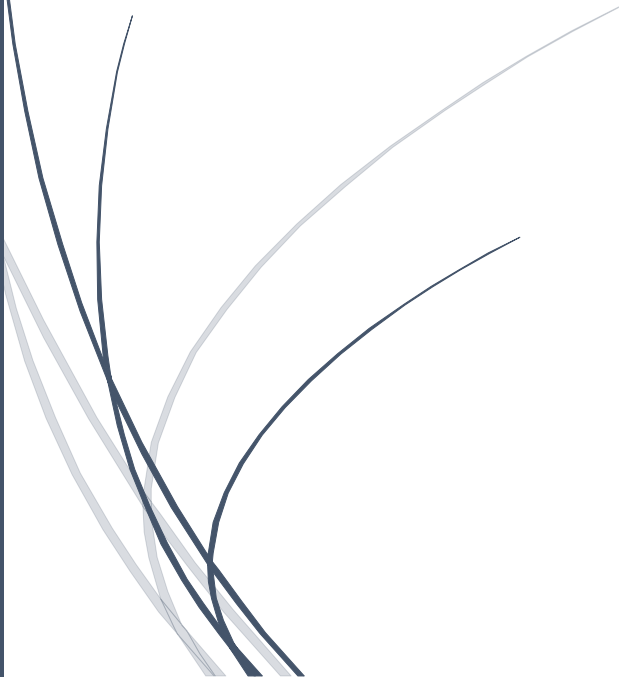


18-3-2021

UPA

Mongo DB

Basico Intermedio



Contenido

Information	2
JavaScript	3
Backup and Restore	3
DDL	4
1. Create	4
2. Drop	5
3. Alter	5
CRUD	6
1. Insert	6
2. Delete	7
3. Update	8
4. Select	9
Aggregate	11
Inner join	13
Arreglos	14
Funciones	15
Cursorres	16
Permisos	¡Error! Marcador no definido.

Information

No.	S Q L	MongoDB
		DB, Colecciones, Documentos (Objetos Json) DB, Tablas, Renglones
	Path	C:\Program Files\MongoDB\Server\5.0\bin
	Version	C:\Program Files\MongoDB\Tools\100\bin c:\>mongod --version c:\>mongo --version
	Crear Directorios para el funcionamiento de mongod	c:\>mkdir data c:\data>mkdir db
	Servidor	c:\>mongod
	Cliente	c:\>mongo
	Conexion	C:\> mongo mongodb://localhost:27017/Mexico C:\> mongo mongodb://localhost:27017/mydb -u user -p password
	show databases	show dbs default (admin, config, local)
	Muestra la DB activa	db
	Crear o usar la DB mydb	use mydb
	Comandos Generales	help
	Comandos de db	db. <tab> <tab>
		db.help()
	Mostrar las Colecciones	show collections db.getCollectionNames()
	Mostrar comandos parecidos	db.ciudades.insert <tab> <tab>
	Crea si no existe Colecciones	db.ciudades.insertOne({idciudad: 33, nombre: "Foraneos"})
	Comandos de Collections	db.ciudades. <tab> <tab>
		db.ciudades.help()
	Contar los que comienzan con “F”	db.ciudades.count({nombre: /^F/})
	Explain	db.ciudades.find({municipios:5}).explain("executionStats")

JavaScript

No.	S Q L	MongoDB
		Math.pow(2,3)
		"Hello World!".replace("World", "Mongodb")
	Funciones	var fnDoble = function (n) {return n*n} fnDoble(5)
		var stuff = [2,3,4,5] for (element of stuff) {print(element*2)}
		var x = {nombre:"Pepito", edad: 20} x.nombre

Backup and Restore

No.	S Q L	MongoDB
	Crear Respaldo Crear una carpeta dump/mydb	C:\> mongodump --db mydb
	Restaurar respaldo Dropear todas las colecciones	C:\> mongorestore --db mydb dump/mydb
	Respaldar collection en dump/mydb	C:\> mongodump --collection Ciudades --db mydb
	Restaurar collection Pero antes borrar coleccion ciudades	C:\> mongorestore --collection Ciudades --db mydb dump/mydb/Ciudades.bson

DDL

1. Create

No.	S Q L	MongoDB
	Create DB	Use Mexico
	Create Collection	db.createCollection("Empleados") use Empleados
	Check o Validar	db.createCollection("Empleados", { validator: { \$and: [{ Nombre: { \$type: "string" } }, { Sexo: { \$in: ["H", "M"] } }, { Fec_Nac: { \$type: "date" } }, { Email: { \$regex: /@/ } }, { Hijos: { \$type: "number" } }] } })
	Create Collection e Insert	db.Empleados.insert({ Nombre: "Pepito", Sexo: "H", Fec_Nac: new Date("2021/03/17"), Email: "pepito@upa", Hijos: 2 })
	Crear indices	db.Orders.createIndex({OrderID:1})
	Mostrar Indices	db.Empleados.getIndexes();

2. Drop

No.	S Q L	MongoDB
	Drop DB	db.getSiblingDB("webstore").dropDatabase();
	Drop DB actual	db.dropDatabase()
	Drop Collection	db.getSiblingDB("testdb").getCollection("cars").drop(); db.products.drop()
	Eliminar Columna	db.Empleados.update({}, {\$unset: {Telefono:1}} , {multi: true});
	Drop Indices	db.Orders.dropIndex({OrderID:-1})

3. Alter

No.	S Q L	MongoDB
	Renombrar Coleccion	db.Academicos.renameCollection("Maestros");
	Todas las columnas	db.ciudades.updateMany({},{\$rename: {"abreviatura": "abr"}});
	Renombrar Columna del primero encontrado	db.ciudades.update({}, { \$rename: { "Abreviatura": "Abr" } }, {multi:true})

CRUD

1. Insert

No.		S Q L	MongoDB
			<pre>[var] doc = {username : "Pablo Milanes", age: 60} db.users.insert(doc)</pre>
			<pre>db.products.insert({ "nombre": "laptop", "precio": 40.2, "active": false, "created_at": new Date("2021/03/17"), "somedata": [1, "a", []], "facturer": { "name": "dell", "location": { "city": "usa", "address": "known" } } })</pre>

2. Delete

No.	S Q L	MongoDB
	Truncate coleccion	db.users.remove({})
	Eliminar documento	db.users.remove({"username": "Luis"})
		db.ciudades.deleteOne({_id: ObjectId("6197d9e68ba4caf6f940cb04")});
		db.ciudades.deleteOne({IdCiudad: 33});

3. Update

No.	S Q L	MongoDB
	Reemplaza el documento o registro completo	<code>db.products.update({"name":"keyboard"}, {"price":99.99})</code>
	Si no existe el atributo, lo agrega	<code>var usuario = db.users.findOne({"username":"Luis"}) usuario.age = 25 db.users.save(usuario)</code>
	Un solo registro	<code>var usuario = db.users.findOne({"username":"Luis"}) print(usuario) usuario.cp = 30000 db.users.update({"username":"Luis"}, usuario)</code>
	Upsert, Si no existe inserta registro Modifica los campos deseados	<code>db.ciudades.update({ IdCiudad: 31}, { \$set: { "Capital": "Meridiano" } }, { upsert: true, multi:true})</code>
	Update ciudades set mun=mun+1 Where idciudad = 31;	<code>db.ciudades.update({ IdCiudad: 31},{ \$inc:{Municipios:1}})</code>

4. Select

No.	S Q L	MongoDB
	Select * from ciudades Order by id desc	db.ciudades.find({}) .projection({Capital:1, _id:0}) .sort({_id:-1}) .limit(100)
		db.Ciudades.findOne()
		db.Ciudades.find({Abreviatura: /^C/})
		db.Ciudades.count({Nombre: /^C/}); db.Ciudades.find({Nombre: /^C/}).count();
	Select * from ciudades Where mun > 50 And nombre like "C%"	db.ciudades.find({ Municipios: { \$gt: 50 }, Nombre : /^C/ });
	Select * from ciudades Where Mun > 100 Or Nom = "Aguascalientes"	db.ciudades.find({ "\$or": [{ Municipios: { \$gt: 100 } }, { Nombre : "AGUASCALIENTES" }] });
	Select * from ciudades Where Nombre = Capital	db.ciudades.find({ "\$where": "this.Nombre == this.Capital" });
	select * from Ciudades where Mun > 50 or Nombre != Capital	db.ciudades.find({ "\$or": [{ Municipios: { \$gt: 50 } }, { "\$where": "this.Nombre != this.Capital" }] });
	select * from Ciudades where Municipios > 100 and Nombre != Capital and Abreviatura != "Mex" Order by Capital desc Limit 3	db.ciudades.find({ Municipios: { \$gt: 100 }, "\$where": "this.Nombre != this.Capital", Abreviatura: { \$ne: "Mex" } }).sort({ Capital: -1 }).skip(1).limit(4).pretty()
	Mostrar ciudades que tengan el campo de Gobernador	db.ciudades.find({"Gobernador": {\$exists: true}})
	Select IdCiudad, Abr from ciudades Where abr not in ("Ags", "Zac")	db.ciudades.find({Abreviatura: {\$nin:["Ags", "Zac"]}}, {_id:0, IdCiudad:1, Abreviatura:true});

	Select * from Orders Where orderdate between x and y	<pre> db.Orders.find({ OrderDate: { \$gte: new Date("1998-04-21"), \$lt: new Date("1998-04-22") } }) </pre>
	Select * from orders Where year(OrderDate) = 1997	<pre> db.Orders.find({ "\$expr": { "\$eq": [{ "\$year": "\$OrderDate" }, 1997] } }) </pre>

Aggregate

N o.	S Q L	MongoDB
	Select count(distinct EmployeeID) From Orders;	db.Orders.distinct("EmployeeID").length
	SELECT distinct IdCiudad FROM ciudades Order by IdCiudad desc	db.ciudades.aggregate([{\$group: {_id: {IdCiudad: "\$IdCiudad"}}}, {\$project: {IdCiudad: "\$_id.IdCiudad" , _id: 0}}, {\$sort: { IdCiudad :-1}}])
	Select IdCiudad, count(*) From ciudades Group by IdCiudad Order by count(*) desc	db.ciudades.aggregate([{\$group: {_id: "\$IdCiudad", "Conteo": { \$sum: 1 }}}, {\$sort: {Conteo:-1}}])
	Select year(OrderDate), count(*) From Orders Group by year(OrderDate)	db.Orders.aggregate([{ \$group: { _id: { year: { \$year: "\$OrderDate" }}, conteo: {\$sum: 1}}}, { \$sort: {"_id.year":1}}])
	Select year(OrderDate), month(OrderDate), count(*) From Orders Group by year(OrderDate), month(OrderDate) Order by 1,2	db.Orders.aggregate([{ \$group: { _id: { year: { \$year: "\$OrderDate" }, Mes: {\$month: "\$OrderDate"}}}, conteo: {\$sum: 1}}}, { \$sort: {"_id.year":1, "_id.Mes":1 } }])
	Select OrderId, sum(UnitPrice*Quantity) From OrderDetails Group by OrderID Order by OrderId	db.OrderDetails.aggregate({ \$group: { _id: "\$OrderID", Importe: { \$sum: { \$multiply: ["\$UnitPrice", "\$Quantity"] } } } }).sort({"_id": 1})
	select sum(Quantity*UnitPrice*(1-Discount)) Total FROM OrderDetails	db.OrderDetails.aggregate([\$group: { _id: null, Total: { \$sum: { \$multiply: [{ \$multiply: ["\$UnitPrice", "\$Quantity"]

		<pre> }, { \$subtract: [1, "\$Discount"] }] } } } })</pre>
<pre>select OrderID, sum(Quantity*UnitPrice*(1-Discount)) Total FROM OrderDetails Group by OrderID Order by OrderID</pre>	<pre>db.OrderDetails.aggregate({ \$group: { _id: "\$OrderID", Total: { \$sum: { \$multiply: [{ \$multiply: ["\$UnitPrice", "\$Quantity"] }, { \$subtract: [1, "\$Discount"] }] } } } }).sort("_id":1)</pre>	
<pre>select idCiudad, count(*) from Municipios where idCiudad != 31 group by idCiudad having count(*) >100 order by count(*);</pre>	<pre>db.Municipios.aggregate([{\$match: {IdCiudad: {\$ne: 31}}}, {\$group: {_id: "\$IdCiudad", "Conteo": { \$sum: 1 }}}, {\$match: {"Conteo": {\$gt:100}}}, {\$sort: {"Conteo":1}}])</pre>	

Inner join

No.	S Q L	MongoDB
	<pre>Select C.IdCiudad, Abr, Nombre_Mun from Ciudades C inner join Municipios M on C.IdCiudad = M.IdCiudad where C.IdCiudad = 1</pre>	<pre>db.Ciudades.aggregate([{\$match: {IdCiudad:1}}, {\$lookup: {from: "Municipios", localField:"IdCiudad", foreignField:"IdCiudad", as: "Mun"}}, {\$unwind: "\$Mun"}, {\$project: {_id:0, IdCiudad:1, Abr:1, Municipio: "\$Mun.Nombre_Mun"}}])</pre>
	<pre>Select IdCiudad, Abr, Count(*), sum(Localidades) From Ciudades C, Municipios M Where C.IdCiudad = M.IdCiudad And C.IdCiudad = 1 Group by IdCiudad, Abr</pre>	<pre>db.Ciudades.aggregate([{\$match: {IdCiudad:1}}, {\$lookup: {from: "Municipios", localField:"IdCiudad", foreignField:"IdCiudad", as: "Mun"}}, {\$unwind: "\$Mun"}, {\$group: { _id: { IdCiudad: "\$IdCiudad", Abr: "\$Abr" }, Tot_Mun: { \$sum: 1 }, Tot_Loc: { \$sum: "\$Mun.Localidades" } }}])</pre>

Arreglos

No.	S Q L	MongoDB
	Los vectores comienzan en la posicion 0.	<pre>db.usuarios.drop() var arreglo= [11,2,3] var usuario = {Nombre:"Carlos", valores: arreglo} db.usuarios.insert(usuario)</pre>
	Agregar sin duplicados	<pre>db.usuarios.update({}, {\$addToSet: {valores: 3}})</pre>
	Agregar con duplicados	<pre>db.usuarios.update({}, {\$push: {valores: 3}})</pre>
	Agregar varios	<pre>db.usuarios.update({}, {\$push: {valores: {\$each: [3,4]}}})</pre>
	Agregar a partir de la posicion	<pre>db.usuarios.update({}, {\$push: {valores: {\$each: [10,20], \$position: 1 }}})</pre>
	Agregar y ordenar desc	<pre>db.usuarios.update({}, {\$push: {valores: {\$each: [10,20], \$sort:-1}}})</pre>
	Ordenar	<pre>db.usuarios.update({}, {\$push: {valores: {\$each: [], \$sort:-1}}})</pre>
	Eliminar el Numero	<pre>db.usuarios.update({}, {\$pull: {valores:11}})</pre>
	Eliminar por condicion	<pre>db.usuarios.update({}, {\$pull: {valores:{ \$gt:10}}})</pre>
	Eliminar lista	<pre>db.usuarios.update({}, {\$pullAll: {valores:[11,3]}})</pre>
	Eliminar la posicion 0	<pre>var data = db.usuarios.findOne({"Nombre":"Carlos"}) data.valores.splice(0,1) db.usuarios.save(data)</pre>
	Regresa la posicion	<pre>usuario.valores.indexOf('11') ??</pre>
	Seleccionar elemento	<pre>db.usuarios.find({},{valores:{\$slice:-1}})</pre>
	Seleccionar rango	<pre>db.usuarios.find({},{valores:{\$slice:[0,2]}})</pre>

Funciones

No.	S Q L	MongoDB
		function factorialR(n) { if (n <= 1) return 1; return n*factorialR(n-1) }
	between	db.products.remove({precio: {\$gt: 1, \$lt:99 }})

Cursores

No.	S Q L	MongoDB
		db.Ciudades.find().forEach(ciudad => print("Ciudad: " + ciudad.IdCiudad, "\t", ciudad.Abreviatura, "\t", ciudad.Nombre))
		for (i = 0; i < 5; i++) { db.products.insert({ "precio": i }) }
	Update 1X1 Cursor es valido solo una vez	var cursor = db.products.find() cursor.forEach((it) => { it.precio = it.precio + 1; db.products.save(it); });
	Update 1X1	var cursor = db.products.find() cursor.forEach(function(it) { it.precio = it.precio + 1; db.products.save(it); })

Otros

No.	S Q L	MongoDB
	Buscar en Arreglos	db.Restaurant.find({"grades.grade": "Z", "grades.score":20}).count()
	\$elemMatch en la misma posicion	db.Restaurant.find({grades: {\$elemMatch: {grade:"Z", score: 20}}}).count()
	Muestra solo el grado	db.Restaurant.find({"grades.grade":"Z"}, {_id:0, "grades.\$":1})